Review Report

LibRA – Library Management System

Project	LibRA – Library Management System (Kütüphane Yönetim Sistemi)
Reviewers	[Reviewer names to be filled]
Review Date	March 2026

Section	Evaluation
1. Methodology Under Review	LibRA, Katmanlı Mimari (Layered Architecture) üzerine inşa edilmiş web tabanlı bir kütüphane yönetim sistemidir. Mimari dört ana katmandan oluşmaktadır: Sunum Katmanı (Presentation Layer – HTML/CSS/JS + Spring Boot Controller), Servis Katmanı (Service Layer – iş mantığı), Veri Erişim Katmanı (Repository Layer – Spring Data JPA) ve Veri Katmanı (MySQL). Tanımlanan senaryolar: Kullanıcı kimlik doğrulama ve rol tabanlı erişim (Admin/Üye), kitap arama (Strategy Pattern ile), kitap ödünç alma/iade, gecikme cezası hesaplama, bekleme kuyruğu (waitlist) yönetimi ve kullanıcı profil görüntüleme. Tasarım örüntüleri: Strategy Pattern (ISearchStrategy arayüzü ile farklı arama stratejileri), Repository Pattern (Spring Data JPA), Layered Architecture (katmanlar arası sorumluluk ayrımı).
2. Technological Context	Gereksinimler: 10 fonksiyonel gereksinim (kullanıcı kimlik doğrulama, rol tabanlı erişim, kitap yönetimi, çok amaçlı arama motoru, kişiye özel tavsiyeler, profil sayfası, ödünç alma, iade takvimi, gecikme cezası hesaplama, bekleme kuyruğu) ve 5 fonksiyonel olmayan gereksinim (hızlı yanıt süresi, kullanılabilirlik, hata toleransı, uyumluluk, veri güvenliği) tanımlanmıştır. Seçilen tasarım örüntüleri: Strategy Pattern (arama motorunda farklı arama tiplerini soyutlamak için ISearchStrategy → TitleSearchStrategy, AuthorSearchStrategy, GeneralSearchStrategy, CategorySearchStrategy), Repository Pattern (Spring Data JPA arayüzleri), MVC (Controller-Service-Repository katmanlaşması). Teknoloji yığını: Java (Spring Boot), Spring Data JPA, MySQL, HTML/CSS/JavaScript (Frontend), IntelliJ IDEA / VS Code, GitHub.
3. Summary of the Method	LibRA, Java Spring Boot üzerine inşa edilmiş katmanlı mimariye sahip bir web tabanlı kütüphane yönetim sistemidir. Sistem, kullanıcıların kitap arayabildiği, ödünç alabildiği, iade edebildiği ve kategori bazlı öneriler alabildiği bir platform sunar. Controller-Service-Repository katmanlaşması ile sorumluluklar net biçimde ayrılmıştır. Arama motoru, Strategy Pattern kullanarak farklı arama tiplerini (başlık, yazar, kategori, genel) bağımsız sınıflara delege eder. Gecikme cezası hesaplama ve bekleme kuyruğu (waitlist) mekanizmaları servis katmanında iş mantığı olarak implement edilmiştir.

4. System Evaluation	Sistem, katmanlı mimari açısından iyi tanımlanmıştır. Controller, Service, Repository ve Entity katmanları arasındaki sorumluluk dağılımı açıkça belirlenmiştir. Class diagramları (ilk versiyon, Strategy Pattern eklentisi ve son versiyon) sistemin evrimini göstermektedir. Roller (Admin ve Üye) tanımlanmış, ancak yetkilendirme mekanizmasının nasıl enforce edileceği (Spring Security, JWT, session-based vb.) tasarım dokümanında detaylandırılmamıştır. Bu, önemli bir mimari boşluktur. Servisler arası ilişkiler (BorrowingService → PenaltyService, BorrowingService → WaitlistService) tanımlanmış olsa da, bu bağımlılıkların yönetimi (dependency injection, circular dependency riski) eksik bırakılmıştır. Sequence diagram yalnızca tek bir senaryo (ödünç alma) için metinsel olarak anlatılmış; diğer kritik akışlar (login, iade, ceza hesaplama) için sequence diagram eksiktir.
5. Assumption Validity	Geçerli varsayımlar: Tek makine (single node) deployment modelinin öğrenci projesi ölçeğinde yeterli olacağı varsayımı doğrudur. Spring Boot + JPA + MySQL kombinasyonunun CRUD ağırlıklı bir kütüphane sistemi için uygun olacağı varsayımı geçerlidir. Ekip üyelerinin haftalık 2–3 saat ayırabileceği varsayımı sınav haftaları dışında makul düzeydedir. Sorunlu varsayımlar: Toplam 100 kişi-saat efor tahmini, 10 fonksiyonel gereksinim + frontend + test + dokümantasyon için yetersiz görünmektedir. Özellikle “İleri Seviye Özellikler” fazına yalnızca 8 saat ayrılması, ödünç alma, ceza hesaplama ve bekleme kuyruğu gibi karmaşık iş mantıkları için çok düşüktür. “Kullanıcıların %60’ı uygulamayı kullanışlı bulmalı” başarı kriteri ölçülmesi güç bir metriktir; nasıl test edileceği belirtilmemiştir. Frontend teknolojisinin henüz kesinleşmemiş olması (“HTML/CSS/JS veya JavaFX” ifadesi) tasarım aşamasında bir belirsizlik yaratmaktadır.
6. Identified Weaknesses	Güvenlik tasarımı eksikliği: Rol tabanlı erişim kontrolu bir gereksinim olarak belirtilmiş, ancak tasarım dokümanında authentication/authorization mekanizması (Spring Security, JWT, session yönetimi) detaylandırılmamıştır. Veri güvenliği (NFR: şifreleme) gereksinimine rağmen şifreleme stratejisi (BCrypt vb.) belirtilmemiştir. Frontend mimarisi belirsizliği: Proje planında “HTML/CSS/JS veya JavaFX” ifadesi geçmektedir; tasarım dokümanında HTML/CSS/JS olarak kesinleştirilmiş görünse de, frontend’in Spring Boot ile nasıl entegre olacağı (Thymeleaf, REST API + SPA, veya statik dosyalar) netliğe kavuşturulmamıştır. Efor tahmininin dengesizliği: Temel geliştirme fazına 40 saat, ileri seviye özelliklere ise yalnızca 8 saat ayrılmıştır. Ödünç alma, ceza hesaplama ve waitlist gibi iş mantıkları en az temel CRUD kadar karmaşıktır. Gereksinim tanımlarının form eksikliği: Fonksiyonel gereksinimler “The system shall...” formatında yazılmamış, maddeler halinde genel ifadeler kullanılmıştır. Test edilebilirlik kriterleri (kabul kriteri, sınır değerleri) belirtilmemiştir. Test stratejisi yeterince detaylandırılmamıştır: Test fazına 10 saat ayrılmış, ancak hangi test türlerinin (unit, integration, UI) uygulanacağı, test araçlarının (JUnit, Mockito, Selenium) neler olacağı belirtilmemiştir.

7. Missing Mechanisms	Authentication/Authorization mekanizması: Spring Security veya JWT tabanlı bir kimlik doğrulama ve yetkilendirme altyapısı tasarımda tanımlanmamıştır. Rol tabanlı erişim kontrolünün endpoint düzeyinde nasıl enforce edileceği belirsizdir. Hata yönetimi (Exception Handling): Controller ve Service katmanlarında global exception handling stratejisi (@ControllerAdvice, custom exception sınıfları) tanımlanmamıştır. NFR olarak “hata toleransı” belirtilmiş ancak mimari düzeyde çözüm sunulmamıştır. Veri doğrulama (Validation): Girdi doğrulama mekanizması (Bean Validation, @Valid) tasarımda yer almamaktadır. Transaction yönetimi: Ödünç alma işleminde stok azaltma + kayıt oluşturma gibi atomik olması gereken işlemlerin transactional yönetimi belgelenmemiştir. API dokümantasyonu: REST endpoint’lerinin detaylı tanımı (HTTP method, URL pattern, request/response format) eksiktir. Concurrency kontrolü: Aynı kitabı aynı anda iki kullanıcının ödünç almaya çalışması durumunda stok tutarlılığının nasıl sağlanacağı (optimistic/pessimistic locking) tanımlanmamıştır.
8. Scenario Evaluation	Senaryo: Bir öğrenci, dönem ödevi için belirli bir kitabı aramak, stokta varsa ödünç almak, stokta yoksa bekleme listesine girmek ve iade süresini takip etmek istiyor. Mevcut davranış: Öğrenci sisteme kayıt olup giriş yapabilir (UserController + UserService). Arama motorunu kullanarak kitabı başlık, yazar veya kategoriye göre bulabilir (BookService + ISearchStrategy). Stokta varsa ödünç alabilir (BorrowingController + BorrowingService). Stokta yoksa bekleme kuyruğuna girebilir (WaitlistController + WaitlistService). Profil sayfasından mevcut ödünç durumunu görüntüleyebilir (ProfileService). Sorunlu noktalar: (1) Giriş yaptıktan sonra oturum yönetimi ve yetkilendirme mekanizması belirsiz olduğundan, başka bir kullanıcının adına ödünç alma riski güvenlik açısından değerlendirilmemiştir. (2) Aynı anda iki öğrencinin son kalan kitabı ödünç almaya çalışması durumunda stok tutarlılığı garanti altında değildir (concurrency kontrolü yok). (3) Öğrenci kitabı geç iade ettiğinde ceza hesaplama mekanizması tanımlanmış, ancak cezanın ödenmesi veya takibi için bir akış belirsizdir. Pozitif yön: Strategy Pattern sayesinde arama motoru genişletilebilir yapıdadır ve OCP (Open-Closed Principle) uyumu sağlanmıştır. Servisler arası ilişkiler (BorrowingService → PenaltyService → WaitlistService) mantıklı biçimde kurgulanmıştır.
9. Improvement Assessment	Strategy Pattern kullanımı (ISearchStrategy): Arama motorunun genişletilebilirliği için doğru ve iyi uygulanmış bir tasarım kararıdır. Yeni arama filtreleri (ISBN, yayınevi vb.) mevcut kodu değiştirmeden eklenebilir. Class diagram’ın evrimsel sunumu (v1 → v2 → final): Tasarımın iteratif gelişimini göstermesi açısından olumludur ve bilinçli bir mühendislik süreci izlendiğini ortaya koymaktadır. Ancak, önerilen iyileştirmeler dokümanlarda açıkça belgelenmemiştir. Bilinen limitasyonlar ve gelecekte yapılacak işler için ayrı bir doküman veya bölüm bulunmamaktadır. Bu, mühendislik şeffaflığı açısından bir eksikliktir.

	Katmanlı mimarinin seçimi proje ölçeğinde mantıklıdır, ancak katmanlar arası bağımlılık yönetimi (DI, circular dependency önleme) ve cross-cutting concern'ler (logging, exception handling) tasarıma dahil edilmelidir.
10. Implementation Assessment	Class diagramlar tasarım dokümanının temel fikrinin üretken biçimde uygulanabileceğini göstermektedir. Entity sınıfları (Book, User, BorrowingRecord, WaitlistRecord, Penalty), Repository arayüzleri, Service sınıfları ve Controller yapıları katmanlı mimariye uygun biçimde konumlandırılmıştır. Strategy Pattern'ın class diagram'da somut biçimde gösterilmesi (ISearchStrategy arayüzü + 4 concrete strategy sınıfı + BookService içinde Map<String, ISearchStrategy> kullanımı) tasarım örüntüsünün doğru anlaşıldığını göstermektedir. Ancak, tasarım düzeyinde bırakılan boşluklar (authentication, transaction yönetimi, validation) uygulama aşamasında ad-hoc çözümlerle doldurulma riski taşımaktadır. Özellikle güvenlik katmanının tasarımsız implement edilmesi ciddi açıklara yol açabilir. Deployment diagramının tek makine modeli için basitliği uygundur, ancak gerçek bir diyagram yerine metin açıklaması olarak sunulmuştur.
11. Overall Strengths	Strategy Pattern'ın etkin kullanımı: Arama motorunda ISearchStrategy arayüzü üzerinden 4 farklı arama stratejisinin tanımlanması, OCP prensibine uyumu ve tasarım örüntüsünün doğru uygulanması açısından güçlü bir noktadır. Katmanlı mimari seçiminin tutarlılığı: Controller → Service → Repository → Entity zinciri net biçimde tanımlanmış ve class diagramlarda tutarlı biçimde yansıtılmıştır. Tasarımın iteratif evrimi: Class diagram'ın üç versiyonu (temel CRUD → Strategy Pattern eklentisi → tam sistem) tasarımın aşamalı olgunlaştığını göstermektedir. Servisler arası ilişkilerin tanımlanması: BorrowingService → PenaltyService, BorrowingService → WaitlistService bağımlılıkları alan mantığına uygun biçimde kurgulanmıştır. Gerçekçi risk yönetimi: Sınav haftaları, teknik deneyim eksikliği ve GitHub çakışmaları gibi öğrenci projelerine özgü risklerin belirlenmesi ve pratik çözüm önerilerinin sunulması olumludur.
12. Overall Limitations	Güvenlik mimarisinin yokluğu: Projenin en kritik zayıflığıdır. Rol tabanlı erişim kontrolu gereksinim olarak tanımlanmış ancak authentication/authorization mekanizması tasarımda yer almamaktadır. Password hashing stratejisi belirtilmemiştir. Gereksinim tanımlarının yetersiztliği: Fonksiyonel gereksinimler test edilebilir formatta yazılmamış, kabul kriterleri ve sınır değerleri belirtilmemiştir. Bu durum, test fazında neyin "başarılı" sayılacağını belirsiz kılmaktadır. Efor tahmininin dengesizliği: 100 saat toplam efor, projenin kapsamına göre düşük görünmektedir. İleri seviye özelliklere ayrılan 8 saatlik efor özellikle yetersizdir. Eksik UML diyagramları: Deployment diagram görsel olarak sunulmamış, sequence diagram yalnızca tek senaryo için metinsel biçimde verilmiştir. Component diagramın görsel karşılığı dokümanda yer almamaktadır.

	Bilinen limitasyonların belgelenmemesi: Bilinçli basitleştirme veya erteleme kararları için ayrı bir doküman veya bölüm bulunmamaktadır.
13. Recommendation	1. Güvenlik katmanı derhal tasarlanmalıdır: Spring Security + JWT veya session-based authentication entegre edilmeli, password hashing (BCrypt) uygulanmalı, endpoint düzeyinde rol bazlı erişim kontrolü (@PreAuthorize veya SecurityFilterChain) tanımlanmalıdır. 2. Gereksinimler test edilebilir formatta yeniden yazılmalıdır: Her gereksinim için kabul kriteri, sınır değerleri ve beklenen davranış belirtilmelidir. 3. Transaction yönetimi ve concurrency kontrolü tasarlanmalıdır: Ödünç alma gibi atomik işlemlerde @Transactional kullanımı ve stok tutarlılığı için optimistic locking uygulanmalıdır. 4. Eksik UML diyagramları tamamlanmalıdır: Deployment diagram görselleştirilmeli, login, iade ve ceza hesaplama için sequence diagramlar eklenmelidir. 5. Global exception handling (@ControllerAdvice) ve input validation (Bean Validation) mekanizmaları tasarıma dahil edilmelidir. 6. Efor tahmini yeniden değerlendirilmelidir: Özellikle ileri seviye özellikler ve test fazlarına daha fazla zaman ayrılmalıdır.

Accept	Weak Accept	Borderline	Weak Reject	Reject
☐	☐	☑	☐	☐

Section	Evaluation
14. Justification	LibRA projesi, katmanlı mimari seçimi ve Strategy Pattern kullanımı açısından sağlam bir tasarım temeli sergilemektedir. Class diagramların iteratif evrimi (v1 → v2 → final) bilinçli bir mühendislik süreci izlendiğini ortaya koymaktadır. Controller-Service-Repository katmanlaşması proje ölçeğinde doğru ve tutarlıdır. Ancak, projenin birkaç kritik eksikliği “Borderline” değerlendirmesini zorunlu kılmaktadır: (1) Güvenlik mimarisinin tamamen eksik olması, rol tabanlı erişim kontrolü gereksinimiyle doğrudan çelişmektedir. (2) Gereksinim tanımları test edilebilir formatta değildir ve kabul kriterleri yoktur. (3) Efor tahmini, projenin kapsamına göre aşırı iyimserdir. (4) Deployment ve sequence diagramları görsel olarak sunulmamış, component diagram eksiktir. Tasarım, güvenlik katmanının eklenmesi, gereksinimlerin test edilebilir formatta yeniden yazılması ve eksik UML diyagramlarının tamamlanması şartıyla kabul edilebilir seviyeye çıkabilir. Mevcut haliyle “Weak Accept” ile “Weak Reject” arasında bir noktadadır.